Capítulo 1

Iniciando com Prolog

Prof. Esdras Lins Bispo Jr.
bispojr@ufj.edu.br

Lógica para Ciência da Computação
Bacharelado em Ciência da Computação

23 de junho de 2026



Objetivos do Capítulo

Após este capítulo, você será capaz de:

- Escrever e carregar um programa simples em Prolog
- Inserir objetivos no prompt do sistema Prolog (`?-`)
- Compreender a terminologia básica da linguagem
- Distinguir os diferentes tipos de dado (*termos*)

1.1 Iniciando o Prolog

O Prompt do Sistema

Ao iniciar o Prolog, o sistema exibe o **prompt**:

```
?-
```

O prompt indica que o Prolog está **pronto para receber objetivos** (*goals*).

- Cada sequência de objetivos deve ser encerrada com um **ponto final** (`.`)
- A linha só é enviada ao sistema após pressionar **Enter**

> Em diferentes versões do Prolog, o prompt pode variar ligeiramente.

A Primeira Consulta

O predicado `write/1` exibe um termo na saída. O predicado `nl/0` imprime uma nova linha.

CONSULTA

```
?- write('Olá mundo'),nl,write('Bem-vindo ao Prolog'),nl.
```

Executar

Edite o código acima e clique **Executar** para experimentar.

Por Dentro da Consulta

A sequência de objetivos abaixo contém **quatro objetivos** separados por vírgulas (que significam "e"):

```
?- write('Olá mundo'), nl, write('Bem-vindo ao Prolog'), nl.
```

Objetivo	Efeito
`write('Olá mundo')`	Exibe `Olá mundo` na tela
`nl`	Imprime uma nova linha
`write('Bem-vindo ao Prolog')`	Exibe `Bem-vindo ao Prolog` na tela
`nl`	Imprime uma nova linha

Para a sequência **ser bem-sucedida**, cada objetivo é satisfeito em ordem. O sistema responde `true`.

Objetivos e Efeitos Colaterais

Do ponto de vista do sistema Prolog, o que importa é se a sequência de objetivos é **satisfeita ou não**.

- A geração de texto na tela (`write` , `nl`) é um **efeito colateral** (*side effect*)
- O `true` ao final indica que a sequência **teve sucesso**

> Os predicados `write e nl` têm significados **pré-definidos** pelo sistema. Eles são chamados de **predicados embutidos** (*built-in predicates — BIPs*).

> Uma sequência de objetivos inserida no prompt é frequentemente chamada de **consulta** (*query*).

1.2 Programas Prolog

Criando um Programa Prolog

A forma usual de trabalhar é:

1. **Criar** o programa em um editor de texto e salvar como `.pl`
2. **Carregar** o programa com o predicado `consult`
3. **Consultar** a base de dados com sequências de objetivos

```
?- consult('prog1.pl').
```

Se o arquivo existir e for sintaticamente correto, o sistema confirma:

```
# 0.00 seconds to consult prog1.pl
```

> As cláusulas carregadas ficam no **banco de dados Prolog** (*Prolog database*) até o usuário sair do sistema.

Estrutura de um Programa

```
cachorro(fido).
gato(felix).
animal(X) :- cachorro(X).
```

Este programa tem **três cláusulas**, cada uma terminada por ponto final.

Linha	Tipo	Leitura natural
`cachorro(fido).`	**Fato**	"fido é um cachorro"
`gato(felix).`	**Fato**	"felix é um gato"
`animal(X) :- cachorro(X).`	**Regra**	"X é um animal se X é um cachorro"

Fatos e Regras em Detalhe

Fatos — afirmações incondicionalmente verdadeiras:

```
cachorro(fido).    % "fido é um cachorro"
gato(felix).   % "felix é um gato"
```

Regras — a cabeça é verdadeira **se** o corpo for satisfeito:

```
animal(X) :- cachorro(X).
```

- O símbolo `:-` lê-se **"se"**
- `X` é uma **variável** — representa qualquer valor, mas o mesmo nos dois lados

> A partir do fato `cachorro(fido)` e da regra acima, o Prolog pode deduzir que `animal(fido)` é verdadeiro.

Consultando o Banco de Dados

BASE DE CONHECIMENTO

```
cachorro(fido).
gato(felix).
animal(X) :- cachorro(X).
```

CONSULTA

```
?- animal(fido).
```

Executar

Experimente também: `animal(felix).` — e observe a diferença no resultado.

Sucesso e Falha

Dizemos que um objetivo é **satisfeito** (*succeeds*) ou **falha** (*fails*):

```
?- animal(fido).
true              % há evidência de que fido é um cachorro → animal

?- animal(felix).
false               % felix é gato, mas não há regra que o torne animal
```

Um objetivo pode ser interpretado como:

- **Comando**: `?- halt.` → *"Sair do Prolog"*
- **Pergunta**: `?- animal(fido).` → *"fido é um animal?"*

Programa 1 - Animais

```
cachorro(fido).    cachorro(rover).
cachorro(tom).     cachorro(henry).
gato(maria).    gato(harry).
gato(bill).    gato(steve).
```

Quatro cláusulas definem o predicado `cachorro` e quatro definem `gato`.

```
?- cachorro(fido).
true

?- cachorro(daisy).
false
```

Variáveis em Consultas

Quando uma consulta contém uma **variável**, o Prolog busca um valor que a satisfaça:

BASE DE CONHECIMENTO

```
cachorro(fido).
cachorro(rover).
cachorro(tom).
cachorro(henry).
```

CONSULTA

```
?- cachorro(X).
```

Executar

O componente retorna a **primeira** solução. No Prolog interativo, pressionar `;` força o sistema a buscar a próxima.

Retrocesso — *Backtracking*

Quando há múltiplas soluções para um objetivo, o Prolog usa **retrocesso** (*backtracking*):

```
?- cachorro(Y).
Y = fido ;      ← usuário pressiona ;
Y = rover ;     ← usuário pressiona ;
Y = tom ;       ← usuário pressiona ;
Y = henry       ← sem mais soluções
```

- O nome da variável (`X` ou `Y`) é indiferente
- O Prolog sempre encontra soluções **na ordem em que as cláusulas foram carregadas**

Combinando Objetivos com Variáveis

BASE DE CONHECIMENTO

```
cachorro(fido).
cachorro(rover).
cachorro(tom).
cachorro(henry).
```

CONSULTA

```
?- gato(X), cachorro(Y).
```

Executar

Esta consulta retorna a **primeira** combinação possível de gato e cachorro.

Experimente também: `gato(X), cachorro(X).` — por que o resultado é diferente?

Mesma Variável, Mesmo Valor

```
?- gato(X), cachorro(Y).   % X e Y são independentes → combinações livres
X = maria, Y = fido

?- gato(X), cachorro(X).   % X deve ser ao mesmo tempo gato E cachorro
false
```

A **mesma variável** em dois objetivos deve ser o **mesmo valor**. Não há animal que satisfaça `gato(X)` e `cachorro(X)` simultaneamente nessa base.

O Predicado `listing`

```
?- listing(cachorro/1).
```

Lista todas as cláusulas que definem o predicado `cachorro`, na ordem em que foram carregadas:

```
cachorro(fido).
cachorro(rover).
cachorro(tom).
cachorro(henry).

true
```

1.3 Objetos de Dados: Termos

Tipos de Termo em Prolog

Os objetos de dados em Prolog são chamados de **termos** (*terms*):

Tipo	Exemplos
Inteiro	`42`, `-7`, `0`, `+5`
Real	`3.14`, `-.5`, `+256.0`
Átomo	`fido`, `'Olá mundo'`, `⩾`
Variável	`X`, `Nome`, `_Temp`
Termo composto	`cachorro(fido)`, `gosta(paul, prolog)`
Lista	`[1, 2, 3]`, `[]`, `[a, [b, c]]`

(1) Números

Inteiros — sequência de dígitos com sinal opcional:

```
623    -47    +5    025
```

Reais — contêm um único ponto decimal:

```
6.43    -.245    +256.0
```

(2) Átomos

Átomos são **constantes sem valor numérico**. Três formas de escrita:

(a) Sequência de letras/dígitos/*underscore* começando com **minúscula**:

```
joao    today_is_Tuesday    fred_jones    a32_BCD
```

(b) Qualquer sequência entre aspas simples (permite espaços e maiúsculas):

```
'Today is Tuesday'    'today-is-Tuesday'    'Olá mundo'
```

© Sequência de caracteres especiais:

```
+++    ⩾    >    +--    :-    =..
```

(3) Variáveis

Em uma consulta, uma variável representa um **termo a ser determinado** pelo sistema.

O nome de uma variável começa com **letra maiúscula** ou *underscore*:

```
X          Author
Pessoa_A   _123A
```

Não são variáveis (começam com minúscula ou dígito):

```
author    45_ABC    pessoa-A
```

> A variável `_` (*underscore* sozinho) é chamada de **variável anônima** e tem propósito especial .

(4) Termos Compostos

Um termo composto começa com um **funtor** (átomo), seguido de **argumentos** entre parênteses:

```
functor(t1, t2, …, tn)    n ≥ 1
```

O número de argumentos é a **aridade** (*arity*).

```
gosta(paul, prolog)                          % aridade 2
cachorro(henry)                                   % aridade 1
pessoa('joao smith', 32, doutor, londres)     % aridade 4
gosta(cachorro(henry), Y)                         % argumento composto
```

> Cada argumento pode ser **qualquer tipo de termo**, inclusive outro termo composto.

(5) Listas

Uma lista é uma sequência de **elementos** (qualquer tipo de termo) entre colchetes:

```
[cachorro, gato, peixe, homem]
[cachorro, gato, y, meupred(A,b,c), [p,q,R], z]
[[joao, 28], [maria, 56, professor], roberto]
```

A **lista vazia** é `[]`.

- Ao contrário da aridade de termos compostos, o número de elementos **não precisa ser definido antecipadamente**
- Um elemento pode ser qualquer termo, inclusive outra lista

Átomos e Termos Compostos: *Call Terms*

Átomos e **termos compostos** têm importância especial nas cláusulas Prolog — são coletivamente chamados de **termos de chamada** (*call terms*).

```
% Átomos usados como predicados (aridade 0):
true.
fail.

% Termos compostos como cabeças de cláusula:
cachorro(fido).
animal(X) :- cachorro(X).
gosta(paul, prolog).
```

Voltaremos a este conceito quando estudarmos a execução de programas.

Resumo do Capítulo 1

- O Prolog usa o prompt `?-` para receber **consultas** (sequências de objetivos)
- Programas são compostos de **fatos** e **regras**, carregados com `consult`
- Um objetivo é **satisfeito** (*true*) ou **falha** (*false*)
- **Variáveis** permitem buscar valores — o Prolog usa **retrocesso** para encontrar todas as soluções
- Os objetos de dados são chamados de **termos**: números, átomos, variáveis, termos compostos e listas